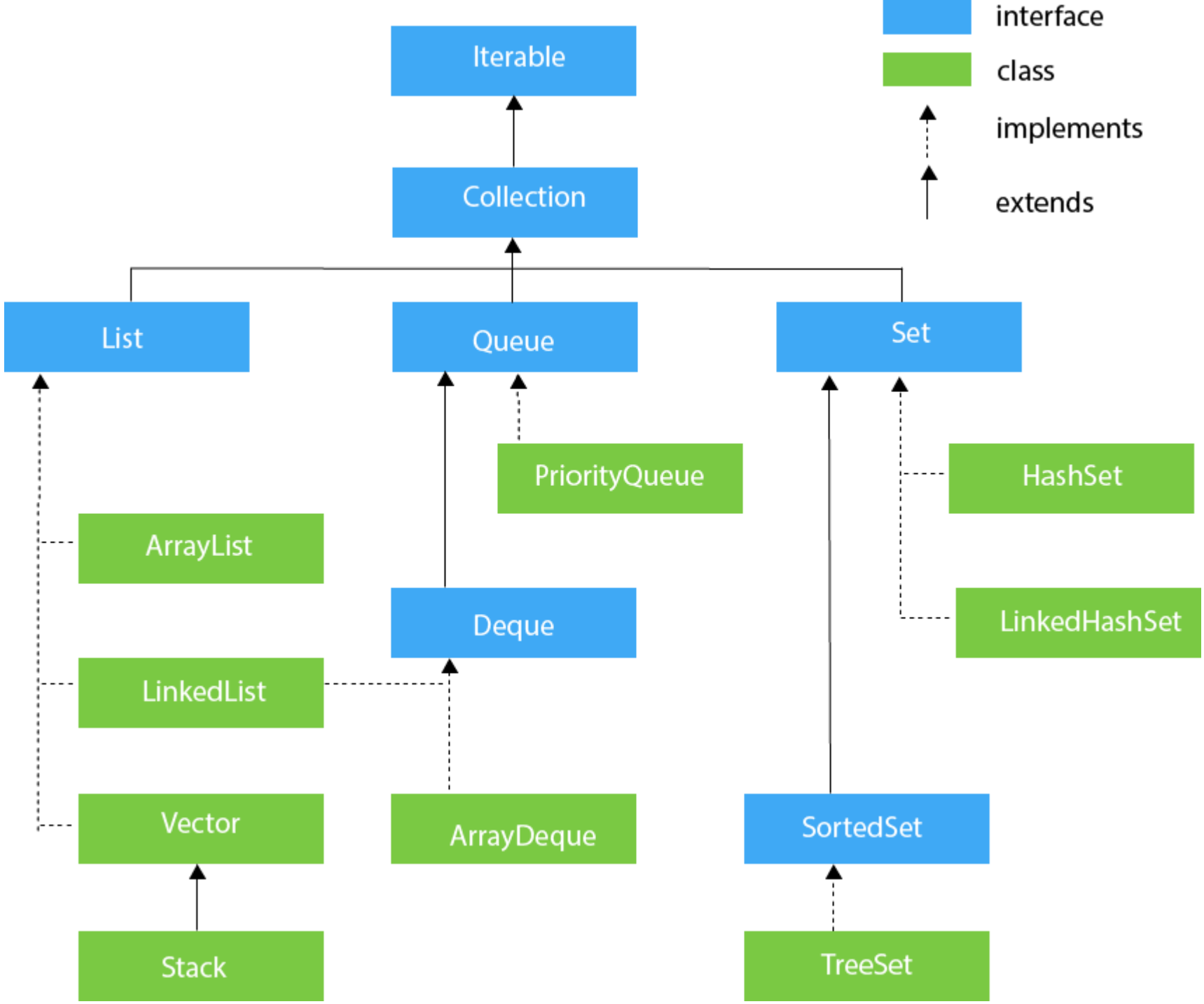
# Collections in Java

- The **Collection in Java** is a framework that provides an architecture to store and manipulate the group of objects.
- Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.
- Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, etc.) and classes (ArrayList, Vector, LinkedList, HashSet,TreeSet etc.).

- What is Collection in Java
- A Collection represents a single unit of objects, i.e., a group.


interface
class
implements
extends
Iterable
Collection
List
Queue
Set
PriorityQueue
HashSet
ArrayList
Deque
LinkedHashSet
LinkedList
Vector
ArrayDeque
SortedSet
Stack
TreeSet

# Methods of Collection interface

- public boolean add(E e)- It is used to insert an element in this collection.
- public boolean addAll(Collection c) - It is used to insert the specified collection elements in the invoking collection.
- public boolean remove(Object element) - It is used to delete an element from the collection.
- public boolean contains(Object element) - It is used to search an element.
- public Iterator iterator() - It returns an iterator.

- Iterator interface

  Iterator interface provides the facility of iterating the elements in a forward direction.
- Methods in the Iterator interface:

✓ public boolean hasNext() - It returns true if the iterator has more elements otherwise it returns false.

✓ public Object next() - It returns the element and moves the cursor pointer to the next element.

✓ public void remove() - It removes the last elements returned by the iterator.

- Any class that implements an Iterable interface, overrides the iterator() method present in the Iterable interface.
- This iterator() method calls an Iterator interface which then returns an iterator.
- This iterator is used to iterate over an object of that class.
- The Iterator interface has methods namely next(), hasNext() and remove().

- Iterable Interface
- The Iterable interface is the root interface for all the collection classes.
- The Collection interface extends the Iterable interface and therefore all the subclasses of Collection interface also implement the Iterable interface.
- It contains only one abstract method. i.e.,
- Iterator<T> iterator()
- It returns the iterator over the elements of type T.

```
import java.io.*;
import java.util.*;
class CollectionDemo {
  public static void main(String[] args)    {
        // Creating instances of the array,
        // vector and hashtable
        int arr[] = new int[] { 1, 2, 3, 4 };
        Vector<Integer> v = new Vector();
        Hashtable<Integer, String> h = new Hashtable();
        // Adding the elements into the
        // vector
        v.addElement(1);
        v.addElement(2);
}
}
```

```
// Adding the element into the
    // hashtable
    h.put(1, "geeks");
    h.put(2, "4geeks");
    // Array instance creation requires [],
    // while Vector and hastable require ()
    // Vector element insertion requires addElement(),
    // but hashtable element insertion requires put()
// Accessing the first element of the
    // array, vector and hashtable
    System.out.println(arr[0]);
    System.out.println(v.elementAt(0));
    System.out.println(h.get(1));

    // Array elements are accessed using [],
    // vector elements using elementAt()
    // and hashtable elements using get()
```

- List Interface
- List interface is the child interface of Collection interface.
- It inhibits a list type data structure in which we can store the ordered collection of objects.
- It can have duplicate values.
- List interface is implemented by the classes ArrayList, LinkedList, Vector, and Stack.

- ArrayList
- The ArrayList class implements the List interface.
- It uses a dynamic array to store the duplicate element of different data types.
- The ArrayList class maintains the insertion order and is non-synchronized(multiple threads can access).
- The elements stored in the ArrayList class can be randomly accessed.

- Java ArrayList class uses a dynamic array for storing the elements.
- It is like an array, but there is no size limit. We can add or remove elements anytime.
- So, it is much more flexible than the traditional array
- Java ArrayList class can contain duplicate elements.
- Java ArrayList class maintains insertion order.
- Java ArrayList allows random access because the array works on an index basis.
- Java ArrayList gets initialized by the size. The size is dynamic in the array list, which varies according to the elements getting added or removed from the list.

# Java For-each Loop or Enhanced For Loop

- It provides an alternative approach to traverse the array or collection in Java.
- It is mainly used to traverse the array or collection elements.
- The drawback of the enhanced for loop is that it cannot traverse the elements in reverse order.
- Here, you do not have the option to skip any element because it does not work on an index basis.
- Moreover, you cannot traverse the odd or even elements only.
- But, it is recommended to use the Java for-each loop for traversing the elements of array and collection because it makes the code readable.

# Linked List

- Java LinkedList class can contain duplicate elements.
- Java LinkedList class maintains insertion order.
- Java LinkedList class is non synchronized.
- In Java LinkedList class, manipulation is fast because no shifting needs to occur.
- Java LinkedList class can be used as a list, stack or queue.

# DOUBLY LINKED LIST

